Tema 1 POO

Responsabil: *Laur Neagu*

MarsGrid – Sistem de Management Energetic pentru o Colonie pe Marte

1. Introducere

Scopul acestei teme este de a simula și gestiona rețeaua energetică a unei colonii autonome pe Marte. Veți modela diverse componente ale rețelei: producători, consumatori și baterii, folosind clase și obiecte.

Tema este concepută pentru a vă testa înțelegerea și aplicarea conceptelor fundamentale POO: **încapsulare**, **abstractizare**, **moștenire** și **polimorfism**. Veți crea ierarhii de clase și veți gestiona interacțiunile dintre obiectele create pentru a menține colonia funcțională.

Scheletul de cod furnizat include teste publice (Test Driven Development) și structura de bază. Urmăriți cu atenție cerințele, acoperind totodată logica centrală a temei – **balansarea energiei la fiecare pas de simulare**.

2. Reguli Generale

1. Nu este permis să modificați numele fișierului sursă unde este metoda main(), folosiți numele furnizat în schelet, și anume Tema1.java
2. Nu este permis să modificați bucata de cod din metoda main() ce face parte din scheletele aplicației. Aceasta va folosi la testare.
3. Nu este permis să modificați nimic (linii de cod, fișiere, foldere) ce se află în directorul src/test, din scheletul de cod.
4. Nu este permis să implementați tema folosind EXCLUSIV generatoare de cod (GitHub Copilot / ChatGPT / CursorAI etc.). La verificarea de plagiat, se va măsura similaritatea codului vostru cu generatoarele de cod.
5. Vă puteți instala și configura Gradle, însă nu este obligatoriu. Prin instalare Gradle, puteți rula aceste teste pe mediile voastre local (rulând comanda: gradle test). La încărcarea codului pe GitHub Classroom (rulând comanda: git push), aceste teste vor rula automat și veți vedea punctajul.

3. Descrierea Problemei

Colonia de pe Marte depinde de o micro-rețea energetică (**MarsGrid**). Sistemul funcționează în **"tick-uri"** (pași de simulare). La fiecare tick, un controller central trebuie să balanseze producția de energie cu consumul. Sistemul va fi controlat de o clasă centrală, GridController, care gestionează liste de componente.

Ierarhia Componentelor:

Veți implementa o clasă de bază abstract class **ComponentaRetea**.

- Atribute comune: String id (unic), boolean statusOperational (dacă funcționează sau e defectă).

1. Producătorii de Energie

- Creați o clasă abstract class **ProducatorEnergie** care **extinde** ComponentaRetea.
- Metodă abstractă: public abstract double calculeazaProductie(double factorExtern);
- Implementări concrete (clase care extind ProducatorEnergie):
  - **PanouSolar**: Producția depinde de factorExtern (procentaj soare). Ex: productie = putereMaxima * factorExtern.
  - **TurbinaEoliana**: Producția depinde de factorExtern (viteză vânt). Ex: productie = putereBaza * factorExtern.
  - **ReactorNuclear**: Producție constantă. Ignoră factorExtern și returnează mereu putereConstanta (dacă statusOperational este true).

2. Consumatorii de Energie

- Creați o clasă abstract class **ConsumatorEnergie** care **extinde** ComponentaRetea.
- Atribute: double cerereEnergie (cât cere pe tick), int prioritate (1-Critic, 2-Normal, 3-Scăzut), boolean esteAlimentat.
- Metodă: public double getCerereCurenta() (returnează cerereEnergie doar dacă esteAlimentat e true, altfel 0).
- Metode: public void cupleazaLaRetea() și public void decupleazaDeLaRetea() (schimbă starea esteAlimentat).
- Implementări concrete (clase care extind ConsumatorEnergie):
  - **SistemSuportViata**: Constructorul va seta prioritate = 1.
  - **LaboratorStiintific**: Constructorul va seta prioritate = 2.
  - **SistemIluminat**: Constructorul va seta prioritate = 3.

3. Stocarea Energiei (Baterii)

- Creați o clasă concretă **Baterie** care **extinde** ComponentaRetea.
- Atribute: double capacitateMaxima, double energieStocata.
- Metode:
  - public double incarca(double energieDisponibila): Încarcă bateria fără a depăși capacitateMaxima. **Returnează surplusul de energie** (energia care nu a avut loc în baterie).
  - public double descarca(double energieCeruta): Descarcă bateria fără a coborî sub 0. **Returnează energia pe care a putut-o furniza**.

Logica Centrală (În GridController)

Clasa **GridController** va avea metoda centrală public void simuleazaTick(double factorSoare, double factorVant). Aceasta este “inima” coloniei și trebuie să urmeze pașii:

1. **Resetare:** Resetează toți consumatorii la starea esteAlimentat = true (la începutul tick-ului, presupunem că toți cer curent).
2. **Calcul Producție:** Calculează productieTotala iterând prin toți producătorii (folosind **polimorfismul** pentru a apela calculeazaProductie() pe fiecare). Se iau în calcul doar cei cu statusOperational = true.
3. **Calcul Cerere:** Calculează cerereTotala iterând prin toți consumatorii (folosind getCerereCurenta()).
4. **Calcul Balansare:** double delta = productieTotala - cerereTotala;

Scenariu 1**: Surplus (delta > 0)**

- Iterează prin lista de Baterii.

- Excedentul (delta) este oferit fiecărei baterii, pe rând, folosind metoda incarca().
- delta devine surplusul returnat de baterie (ce nu a putut stoca). Energia nefolosită se pierde.

Scenariu 2**: Deficit (delta < 0)**
- double deficit = -delta;
- Iterează prin lista de Baterii pentru a acoperi deficitul.
- Apelează descarca(deficit) pe fiecare baterie.
- Actualizează deficit cu energia pe care bateria *nu* a putut-o furniza.

Scenariu 3**: Triage (Dacă deficit încă este > 0 după baterii)**
- Controller-ul trebuie să **decupleze** consumatorii pentru a salva rețeaua.
- **Cerință:** Decuplarea se face în ordinea strictă a priorității: întâi toți cei cu Prioritate 3, apoi toți cei cu Prioritate 2.
- **NU se decuplează NICIODATĂ** consumatorii cu Prioritate 1 (SistemSuportViata).
- *Algoritm recomandat:* Sortează lista de consumatori după prioritate (descrescător, adică 3, 2, 1). Iterează prin lista sortată. Dacă prioritate > 1 și deficit > 0:
  - Apelează consumator.decupleazaDeLaRetea().
  - Scade cerereEnergie a consumatorului din deficit.

Scenariu 4**: Blackout!**
- După Triage (Scenariu 3, dacă deficit este încă > 0 (ceea ce înseamnă că nici producția, nici bateriile nu pot susține nici măcar sistemele critice de Prioritate 1), sistemul intră în **BLACKOUT**.
- GridController va seta o variabilă de stare internă esteInBlackout = true, iar toate simulările viitoare se opresc.

4. Reguli de Implementare

- Folosiți **clase abstracte** (ComponentaRetea, ProducatorEnergie, ConsumatorEnergie) pentru a defini comportamentul comun și a forța implementarea metodelor abstracte în clasele copil.
- Folosiți **încapsularea**: Atributele trebuie să fie private sau protected. Modificarea stării (ex. energieStocata în Baterie) se face doar prin metode publice (incarca/descarca).
- **Polimorfismul** este testat în GridController, care trebuie să stocheze List<ProducatorEnergie> și List<ConsumatorEnergie> și să le poată procesa unitar în bucla de simulare.
- Validarea datelor la adăugare este obligatorie (ex. ID-uri unice, capacitate > 0, prioritate validă).

5. Comenzi Consolă

Aplicația va fi interactivă și va procesa următoarele comenzi:

0. *add_producator <tip> <id> <putere_specifica>*
   - Ex: add_producator solar s1 100 (100W la 100% soare)
   - Ex: add_producator reactor r1 1000 (1000W constant)
1. *add_consumator <tip> <id> <cerere_putere>*
   - Ex: add_consumator suport_viata sv1 800 (Prioritatea 1 e setată automat)
   - Ex: add_consumator laborator lab1 300 (Prioritatea 2 e setată automat)
2. *add_baterie <id> <capacitate_maxima>*
3. *next_tick <factor_soare> <factor_vant>*
   - Declanșează **bucla de simulare (pașii 1-8)**.

- Afișează un rezumat: Producție, Cerere, Energie Baterii (total stocat), Consumatori decuplați (listă de ID-uri).
- Dacă se ajunge la Blackout, afișează "BLACKOUT! SIMULARE OPRITA."

4. *set_defect <id_componenta> <status>*
   - Ex: set_defect r1 false (Setează statusOperational pe false pentru reactorul r1)
   - Ex: set_defect s1 true (Repară panoul s1)
5. *status_grid*
   - Afișează starea detaliată a fiecărei componente (ID, tip, status, energie stocată/cerere) și starea generală a rețelei (Stabil, Deficit, Blackout).
6. *istoric_evenimente*
   - (Simplu) Afișează un istoric al evenimentelor majore (ex. "Tick 5: Deficit - Decuplat lab1", "Tick 10: BLACKOUT"). GridController va trebui să stocheze un List<String> cu aceste mesaje.
7. *exit*

6. Criterii de Evaluare

- **Design-ul Ierarhiei de Clase:** Utilizarea corectă a moștenirii și a claselor abstracte.
- **Încapsulare:** Protejarea datelor și expunerea lor prin metode.
- **Polimorfism:** Procesarea unitară a componentelor (producători, consumatori) în GridController.
- **Corectitudinea Algoritmului de Balansare:** Logica din simuleazaTick (pașii 1-8) este cel mai important criteriu. Se va testa ordinea operațiilor (baterii întâi, apoi triage) și corectitudinea calculelor.

7. Punctaj

DOMENIU	TASK	PUNCTAJ	TOTAL
Teste	Suită de teste rulată automat	80 p	**80 PUNCTE**
Cod	Folosirea conceptelor de POO studiate	5 p	**15 PUNCTE**
	Folosirea structurilor de date studiate	5 p	
	Lizibilitate cod, comentarii (acolo unde este cazul)	5 p	
README	Redactarea unui fișier Readme în care să se detalieze implementarea temei (200-500 cuvinte)	5 p	**5 PUNCTE**

Bonus

Ce alte cazuri limită ați mai trata în această aplicație (minim 3 cazuri, a se descrie în README)? – *5 PUNCTE*

Cum ați refactoriza comenzile și răspunsurile din aplicație (minim 3 propuneri, a se descrie în README)? – *5 PUNCTE*

Pentru Bonus, răspundeți la aceste întrebări în README.
Totodată, pentru a vă maximiza punctajul în cadrul temei, încercați să acoperiți cât mai multe dintre cazurile întoarse de fiecare comandă, și din lanțuri de comenzi succesive.